

CixioHub — Release 1.0 Student Project Guide

Instructor Note: Each student / team builds their own independent version of CixioHub. This document is the single source of truth for what to build, how to build it, and what is expected at the demo.

Table of Contents

1. Project Vision
 2. System Architecture
 3. Tech Stack
 4. Module Specifications
 5. Database Schema
 6. API Reference
 7. Role-Based Assignments
 8. 7-Day Sprint Plan
 9. Environment Setup
 10. Demo & Submission Requirements
 11. Scaling Considerations
-

1. Project Vision

Why are we building CixioHub?

We want to replace ChatGPT with an in-house AI chat platform that runs on our own infrastructure, respects data privacy, and can be customised for institutional use. CixioHub is that platform — built, deployed, and maintained by students of TKM.

Who will use it?

User Type	Description
End Users	All TKM students — access via web browser or mobile app
CET Students	Onboarded in bulk by admin (not self-registration)
Admin / Instructor	Creates bulk accounts, manages users

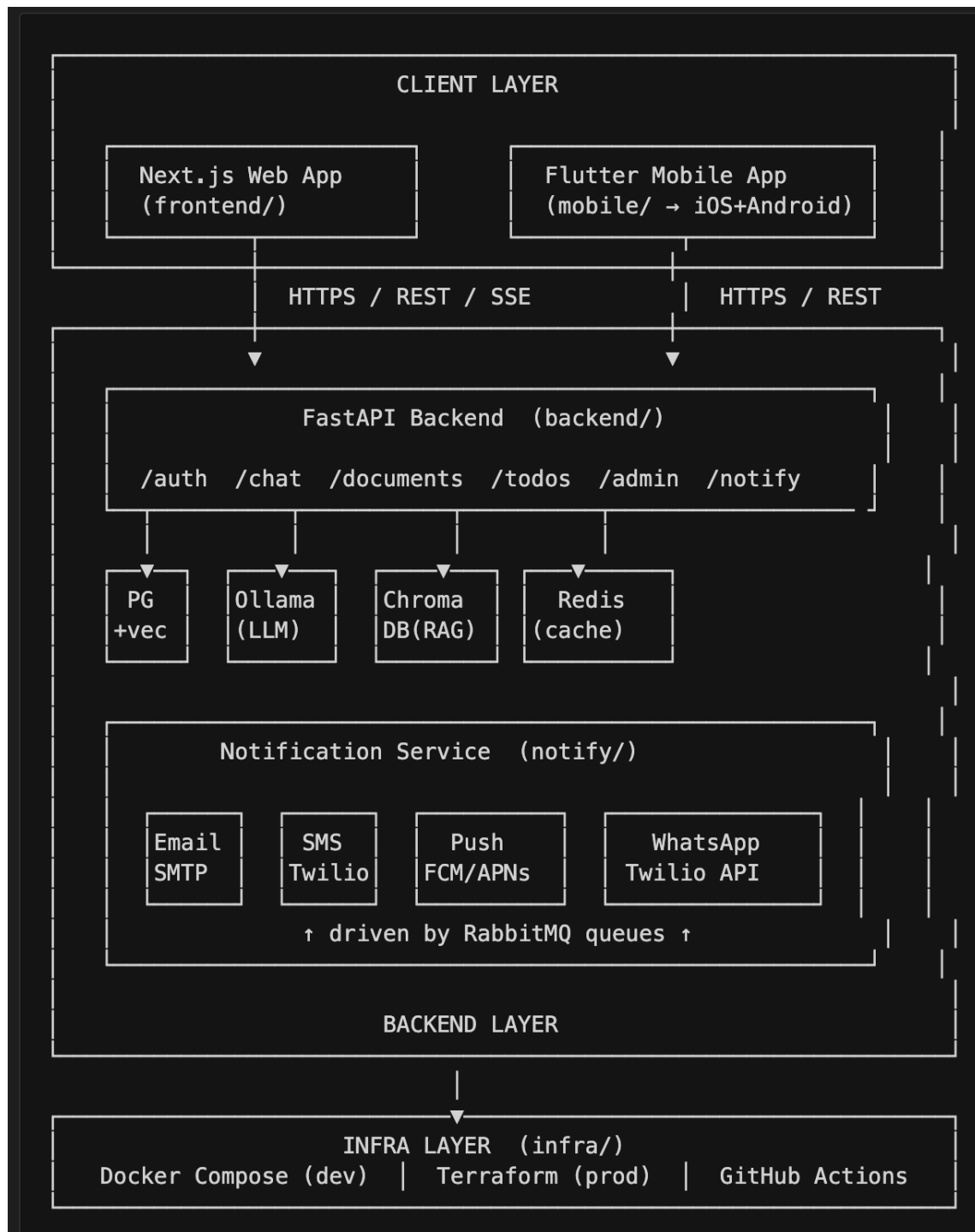
How will it be accessed?

- **Website** — Next.js web application (desktop + mobile browser)
- **Mobile App** — Flutter app targeting both iOS and Android

Onboarding Methods

1. **Self Registration** — TKM students sign up with their email
 2. **Bulk Import (CSV)** — Admin uploads a spreadsheet; system creates accounts and sends temp credentials via email or SMS
 3. **Invite Link** — (Future) shareable invite link
-

2. System Architecture



Communication Patterns

Flow	Protocol
Frontend / Mobile → Backend	REST (JSON) over HTTPS
Chat streaming	Server-Sent Events (SSE)
Backend → Notify service	RabbitMQ message queue
Backend → LLM	HTTP (Ollama local API)
Bulk notifications	RabbitMQ → Notify workers

3. Tech Stack

Layer	Technology	Version / Notes
Mobile	Flutter	3.x — single codebase for iOS & Android
Web Frontend	Next.js	14+ with App Router, TypeScript
Styling	Tailwind CSS	v3
Backend API	FastAPI	Python 3.11+
ORM	SQLAlchemy	2.x + Alembic for migrations
Primary Database	PostgreSQL	16 + pgvector extension
Cache / Sessions	Redis	7.x
Message Queue	RabbitMQ	3.x
Vector Store	ChromaDB	0.5.x — for RAG document search
LLM (local dev)	Ollama	llama3.2:3b (fits on 8 GB RAM)
LLM (production)	Ollama	llama3.1:70b or cloud API

File Storage	AWS S3 (prod) / local filesystem (dev)	
Email	SMTP (dev) / AWS SES (prod)	credentials provided by instructor
SMS	Twilio	credentials provided by instructor
Push Notifications	Firebase FCM (Android) + APNs via AWS SNS (iOS)	
WhatsApp	Twilio WhatsApp API	
Containerisation	Docker + Docker Compose	
IaC	Terraform	AWS provider
CI/CD	GitHub Actions	

4. Module Specifications

There are **8 modules**. You do not need to build all of them — see Role-Based Assignments for what your role covers.

Module 1 — Authentication & User Management

Location: `backend/`

Owner roles: All roles need Auth running; Backend role owns it.

What to build

Feature	Description
Self-registration	User signs up with email + password
Login / Logout	JWT access token + refresh token
Profile management	Edit name, phone, upload avatar (stored in S3 or local)
Bulk user creation	Admin uploads CSV → system creates accounts + sends temp credentials via email/SMS

Google OAuth (optional)	Sign in with Google — nice-to-have
----------------------------	------------------------------------

Rules

- Passwords must be hashed with **bcrypt**
 - Access tokens expire in **30 minutes**; refresh tokens in **7 days**
 - Bulk import must be a background job — do not block the HTTP request
 - Temp credentials should be random 12-character passwords
-

Module 2 — AI Chat (LLM Integration)

Location: `backend/` (API) + AI/LLM team handles the Ollama/RAG layer

Owner roles: AI/LLM, Backend

What to build

Feature	Description
Chat sessions	Each user can have multiple named chat sessions
Send message (streaming)	POST to <code>/chat/sessions/{id}/messages</code> returns SSE stream
Chat history	Retrieve all messages in a session
Context injection	Last N messages sent as context to Ollama
RAG augmentation	If user has uploaded documents, relevant chunks are injected into the prompt
Delete session	Remove a chat session and all its messages

Ollama Integration

- Run Ollama locally: `ollama serve + ollama pull llama3.2:3b`
 - API base URL: `http://localhost:11434`
 - Endpoint: POST `/api/chat` with `stream: true`
 - System prompt: define the assistant persona (e.g., "You are CixioHub, an AI assistant for TKM students.")
-

Module 3 — RAG (Document Upload & Processing)

Location: backend/
Owner roles: AI/LLM

What to build

Feature	Description
Document upload	Accept PDF, DOCX, TXT, PNG/JPG (via OCR)
Text extraction	PyMuPDF (PDF), python-docx (DOCX), Tesseract/Pillow (images)
Chunking	Split text into ~500 token chunks with 50 token overlap
Embedding	Use Ollama embedding model (<code>nomic-embed-text</code>) or <code>sentence-transformers</code>
Store vectors	Save embeddings + chunk text in ChromaDB with <code>user_id</code> metadata
Retrieval	On each chat message, query ChromaDB for top-5 relevant chunks
List / Delete files	User can see their uploaded files and delete them (removes vectors + file)

RAG Prompt Template

```
None

You are CixioHub. Answer the user's question using the context below.
If the context does not help, answer from general knowledge.

Context:
{retrieved_chunks}

User Question: {user_message}
```

Module 4 — Notification Service

Location: notify/
Owner roles: Backend, Frontend, Docker

What to build

Four notification channels:

Channel	Provider	Trigger
Email	SMTP / AWS SES	Account creation, password reset, bulk campaigns
SMS	Twilio	Temp credentials, alerts
Push	Firebase FCM (Android) + APNs via AWS SNS (iOS)	Chat replies, reminders
WhatsApp	Twilio WhatsApp API	Bulk campaigns (optional)

Two send modes:

- **Single send** — `POST /notify/send` — immediate dispatch
- **Bulk send** — `POST /notify/bulk` — publishes to RabbitMQ queue, returns `job_id`

Module 5 — Message Queue

Location: `notify/` (workers) + `backend/` (publisher)

Owner roles: Backend, Docker, Frontend

What to build

Queue Name	Purpose
<code>email.process</code>	Email tasks waiting to be sent
<code>sms.process</code>	SMS tasks
<code>push.process</code>	Push notification tasks
<code>*.retry</code>	Failed tasks waiting for retry (dead-letter queues)
<code>*.failed</code>	Permanently failed after max retries

Retry Logic

- Attempt 1: immediate
- Attempt 2: after 1 minute
- Attempt 3: after 5 minutes
- Attempt 4: after 30 minutes
- After 4 failures: move to `*.failed` queue, update job status

Job Tracking

Track every bulk job in the `notification_jobs` table:

None

total | sent | failed | retrying | completed

Return job status at `GET /notify/jobs/{job_id}`.

Scale Target

- 100,000 emails / day
- 10,000 SMS / day
- System must support 10M+ users with queue backpressure

Module 6 — Frontend Web App

Location: `frontend/`
Owner roles: Frontend, Backend (for API integration)

Pages to build

Route	Page
<code>/login</code>	Login form
<code>/register</code>	Registration form
<code>/chat</code>	Main chat interface
<code>/chat/[sessionId]</code>	Chat session with message history
<code>/documents</code>	File upload + file list
<code>/todos</code>	Todo list
<code>/profile</code>	Edit profile, upload avatar

Chat UI Requirements

- Message thread (user messages on right, AI on left)
- Streaming response — characters appear as they arrive (SSE)
- Session sidebar — list of past chats, create new chat
- Loading indicator while AI is responding
- Code blocks rendered with syntax highlighting
- Markdown rendered (bold, lists, headings)

Non-functional requirements

- Responsive — works on mobile browsers
 - Auth-protected routes — redirect to /login if not authenticated
 - JWT stored in httpOnly cookie or localStorage (cookie preferred)
-

Module 7 — Mobile App (Flutter)

Location: mobile/

Note: One Flutter codebase targets both iOS and Android.

Owner roles: iOS developer, Android developer

Screens to build

Screen	Features
Login	Email + password, "remember me"
Register	Name, email, password
Chat	Message thread, streaming AI response, new chat button
Chat History	List of past sessions
Documents	Upload from phone storage, list, delete
Todos	Create, complete, delete todos
Profile	Edit name, phone, change avatar (camera + gallery)
Settings	Notification preferences, logout

Mobile-specific features

- **Push notifications** — register FCM token (Android) / APNs token (iOS) on login
 - **Deep links** — notification taps open the relevant screen
 - **Offline support** — cache last 10 chat sessions locally (Hive or SharedPreferences)
 - **Camera / gallery** — profile picture upload
-

Module 8 — DevOps / Infrastructure

Location: infra/

Owner roles: Docker team

What to build

Deliverable	Description
<code>docker-compose.yml</code>	Full local stack: all services + databases
Individual Dockerfiles	One per service (backend, frontend, notify, ollama)
Terraform — AWS VPC	Networking, subnets, security groups
Terraform — Compute	ECS Fargate tasks or EC2 instances
Terraform — Data	RDS (PostgreSQL), ElastiCache (Redis), S3
Terraform — Messaging	SNS, SES configuration
GitHub Actions — CI	Lint + test on every PR
GitHub Actions — CD	Build + push Docker image on merge to <code>main</code>
Nginx reverse proxy	Route <code>/api</code> → FastAPI, <code>/</code> → Next.js
<code>.env.example</code>	Document every environment variable

Port Mapping (local Docker Compose)

Service	Port
Next.js (frontend)	3000
FastAPI (backend)	8000
Notify service	8001
ChromaDB	8002
Ollama	11434
PostgreSQL	5432
Redis	6379
RabbitMQ (AMQP)	5672
RabbitMQ (Management UI)	15672

5. Database Schema

All tables live in PostgreSQL. Run migrations with **Alembic**.

None

-- Users

```
CREATE TABLE users (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    email         VARCHAR(255) UNIQUE NOT NULL,  
    phone         VARCHAR(20),  
    full_name     VARCHAR(255) NOT NULL,  
    hashed_password VARCHAR(255) NOT NULL,  
    avatar_url    TEXT,  
    is_active     BOOLEAN DEFAULT TRUE,  
    is_admin      BOOLEAN DEFAULT FALSE,  
    device_tokens TEXT[],          -- FCM / APNs tokens  
    created_at    TIMESTAMPTZ DEFAULT NOW(),  
    updated_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Chat Sessions

```
CREATE TABLE chat_sessions (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id       UUID REFERENCES users(id) ON DELETE CASCADE,  
    title         VARCHAR(255) DEFAULT 'New Chat',  
    created_at    TIMESTAMPTZ DEFAULT NOW(),  
    updated_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Chat Messages

```
CREATE TABLE chat_messages(  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    session_id    UUID REFERENCES chat_sessions(id) ON DELETE CASCADE,  
    role          VARCHAR(20) NOT NULL,    -- 'user' / 'assistant'  
    content       TEXT NOT NULL,  
    created_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Documents

```
CREATE TABLE documents (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id       UUID REFERENCES users(id) ON DELETE CASCADE,  
    filename       VARCHAR(255) NOT NULL,
```

```

        file_type          VARCHAR(50) NOT NULL,    -- 'pdf' | 'docx' |
        'txt' | 'image'
        file_size          INTEGER NOT NULL,         -- bytes
        storage_path       TEXT NOT NULL,
        chroma_collection  TEXT,                    -- ChromaDB collection
name
        processed         BOOLEAN DEFAULT FALSE,
        created_at        TIMESTAMPTZ DEFAULT NOW()
    );

-- Todos
CREATE TABLE todos (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    title       VARCHAR(500) NOT NULL,
    description TEXT,
    completed   BOOLEAN DEFAULT FALSE,
    due_date    TIMESTAMPTZ,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Notification Jobs (bulk sends)
CREATE TABLE notification_jobs (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    channel     VARCHAR(20) NOT NULL,    -- 'email' | 'sms' | 'push' |
    'whatsapp'
    total       INTEGER NOT NULL,
    sent        INTEGER DEFAULT 0,
    failed      INTEGER DEFAULT 0,
    retrying    INTEGER DEFAULT 0,
    completed   BOOLEAN DEFAULT FALSE,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

```

6. API Reference

All endpoints are prefixed with `/api/v1`. Authentication required unless marked **Public**.

Auth

Method	Path	Description	Auth
POST	/auth/register	Create new account	Public
POST	/auth/login	Login, returns JWT tokens	Public
POST	/auth/logout	Revoke refresh token	Required
POST	/auth/refresh	Get new access token	Public (refresh token)
GET	/auth/me	Current user profile	Required
PUT	/auth/profile	Update name, phone	Required
POST	/auth/avatar	Upload profile picture	Required
GET	/auth/google	Start Google OAuth	Public

Admin

Method	Path	Description	Auth
POST	/admin/users/bulk	Upload CSV, create users in background	Admin
GET	/admin/users	List all users	Admin
DELETE	/admin/users/{id}	Delete a user	Admin

Chat

Method	Path	Description	Auth
POST	/chat/sessions	Create new chat session	Required
GET	/chat/sessions	List user's sessions	Required
DELETE	/chat/sessions/{id}	Delete session + messages	Required
POST	/chat/sessions/{id}/messages	Send message (SSE stream)	Required
GET	/chat/sessions/{id}/messages	Get message history	Required

Documents

Method	Path	Description	Auth
POST	/documents/upload	Upload file (multipart)	Required
GET	/documents	List user's documents	Required
DELETE	/documents/{id}	Delete file + vectors	Required

Todos

Method	Path	Description	Auth
GET	/todos	List todos (filter: ?completed=true)	Required
POST	/todos	Create todo	Required
PUT	/todos/{id}	Update title / due date	Required
PUT	/todos/{id}/complete	Mark complete / incomplete	Required
DELETE	/todos/{id}	Delete todo	Required

Notifications

Method	Path	Description	Auth
POST	/notify/send	Send single notification	Required
POST	/notify/bulk	Enqueue bulk notification job	Admin
GET	/notify/jobs	List bulk jobs	Admin
GET	/notify/jobs/{id}	Get job status + progress	Admin

Health

Method	Path	Description	Auth
GET	/health	Service health check	Public

7. Role-Based Assignments

Each student / team picks a role and builds those modules. You may choose 1–2 additional modules from outside your role for bonus scope.

iOS Developer

Must build:

- Flutter mobile app — all screens — targeting iOS *and* Android (one codebase)
- Push notification handling (APNs token registration + FCM)

Choose 1–2 extras from:

- Document upload UI + backend integration
- Todo list backend API
- Google OAuth integration

Demo: Live Flutter app running on iOS Simulator + Android Emulator (or real devices), showing chat, document upload, and push notifications.

Android Developer

Must build:

- Flutter mobile app — all screens — targeting iOS *and* Android (one codebase)
- FCM token registration + notification display

Choose 1–2 extras from:

- Document upload UI + backend integration
- Profile picture with camera
- Offline caching of chat history

Demo: Live Flutter app on Android device or emulator, full chat flow + at least one extra module demonstrated.

Frontend Developer

Must build:

- Next.js web app — all pages
- Full integration with backend API (auth, chat, documents, todos)

Choose 1 from:

- Message queue status page (show bulk job progress)
- Real-time push/in-app notification banner via WebSocket or SSE

Demo: Running Next.js app in browser showing full chat flow end-to-end with a live backend.

Backend / API Engineer

Must build:

- FastAPI backend — all Auth, Chat, Document, and Todo endpoints
- Notification service — all 4 channels (email, SMS, push, WhatsApp)
- RabbitMQ message queue — process, retry, dead-letter queues

Demo: All API endpoints tested via Swagger UI or Postman. Bulk notification job sent, queue consumed, and job status tracked.

AI / LLM Engineer

Must build:

- Ollama integration — chat endpoint with streaming
- RAG pipeline — document upload → extract → chunk → embed → store → retrieve
- Support at least: PDF, DOCX, TXT, and one image format

Choose 1–2 extras from:

- Document Q&A — user can ask questions specifically about one document
- Image analysis — upload an image and ask questions about it
- Summarisation endpoint — one-click summary of an uploaded document

Demo: Live chat session that queries uploaded documents (RAG), with streaming response visible.

Docker / DevOps Team

Must build:

- `docker-compose.yml` — full local stack, all services boot with `docker compose up`
- Dockerfile for: backend, frontend, notify service
- Nginx reverse proxy config
- Terraform — AWS VPC + at least one compute resource (ECS or EC2)
- GitHub Actions CI pipeline — lint + test

Choose 1–2 extras from:

- Full Terraform deployment (RDS + ElastiCache + S3)
- CD pipeline — auto-deploy on merge to `main`
- Multi-region deployment sketch

Demo: `docker compose up` boots the full stack from scratch. All services reachable on correct ports. Show Terraform plan output for AWS.

8. 7-Day Sprint Plan

Day	Goal	All Roles
Day 1	Environment setup	Install tools, clone repos, run <code>docker compose up</code> , pull Ollama model, verify everything starts
Day 2	Auth + skeleton	Auth endpoints working (register, login, JWT). Login screen on web/mobile.
Day 3	Core feature	Backend: Chat API + Ollama stream. Frontend: Chat UI. Mobile: Chat screen. AI: RAG pipeline. Notify: email + SMS channel. DevOps: Dockerfiles.
Day 4	Supporting features	Backend: Documents, Todos. Frontend: Documents + Todo pages. Mobile: Documents + Todo screens. AI: PDF + DOCX support. Notify: push + queue. DevOps: Compose + Nginx.
Day 5	Integration	Connect frontend/mobile to backend. End-to-end: register → login → chat with RAG. Notifications triggered by backend events.
Day 6	Testing + Polish	Fix bugs. Write basic tests. Ensure Docker Compose starts cleanly. Record demo.
Day 7	Demo prep	Record screen capture. Write 1-page summary. Finalise documentation. Send submission email.

Daily Standup (suggested)

Answer these 3 questions each morning:

1. What did I complete yesterday?
2. What am I working on today?
3. What is blocking me?

9. Environment Setup

Prerequisites

Install these before Day 1:

None

1. Docker Desktop

<https://www.docker.com/products/docker-desktop/>

```
# 2. Ollama
https://ollama.com/download
ollama pull llama3.2:3b
ollama pull nomic-embed-text

# 3. Python 3.11+
https://www.python.org/downloads/

# 4. Node.js 20+
https://nodejs.org/

# 5. Flutter 3.x
https://docs.flutter.dev/get-started/install

# 6. VS Code extensions
# - Python, Pylance
# - ESLint, Prettier
# - Flutter, Dart
# - REST Client (for .http files)
# - Docker
```

Local Stack (Docker Compose)

```
None

# From the infra/ directory:
docker compose up -d

# Verify services are running:
docker compose ps
```

This starts: PostgreSQL, Redis, RabbitMQ, ChromaDB.
Ollama and each app service run outside Docker during development.

Backend Setup

```
None

cd backend
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # fill in your values
```

```
alembic upgrade head          # run migrations
uvicorn app.main:app --reload  # start dev server on :8000
```

Open Swagger docs: <http://localhost:8000/docs>

Frontend Setup

```
None
cd frontend
npm install
cp .env.example .env.local      # fill in
NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev                     # start on :3000
```

Mobile Setup

```
None
cd mobile
flutter pub get
cp .env.example .env           # fill in API_BASE_URL
flutter run                    # choose device
```

Notify Service Setup

```
None
cd notify
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload --port 8001
# In a separate terminal, start the queue worker:
python -m app.queue.worker
```

10. Demo & Submission Requirements

What to Demo (1 week from start)

Each demo should show a **working end-to-end flow** relevant to your role. Suggested flow:

1. Register a new account (or log in with a pre-created account)
2. Send a chat message and see a streaming AI response
3. Upload a document and ask a question about it (RAG)
4. Trigger a notification (email or push)
5. Show any additional modules you built

Total demo time: **max 5 minutes** per student/team.

Submission Package

Send via email to the instructor. Include ALL of the following:

Item	Details
GitHub repo link	Your fork/clone with <code>README.md</code> describing setup
Screenshots	Minimum 5 screenshots of working features
Screen recording	Max 5 minutes — demo your core flow
Written summary	1 page: what you built, challenges faced, what you'd improve
<code>.env.example</code>	With all variable names (no real values)

11. Scaling Considerations

Your code should be designed with these scale targets in mind, even if you don't implement full production infrastructure.

Users	What changes
50	Everything on a single machine, SQLite is fine
500	PostgreSQL needed, Redis for sessions
5,000	Add connection pooling, CDN for static assets
100,000	Horizontal scaling of FastAPI workers, RabbitMQ cluster, S3 for files
1,000,000	Read replicas for PostgreSQL, Kafka instead of RabbitMQ, multi-region deployment

Notification scale targets:

- 100,000 emails per day
- 10,000 SMS per day
- Up to 50,000,000 queue messages (assumes 10M users, 5 messages/user)
- Retry budget: assume 10% failure rate → 10M retries need to resolve within 1 hour

Repository Structure

```
None
CixioHub/
├─ backend/           # FastAPI - Auth, Chat, Documents, Todos, Admin
├─ frontend/         # Next.js - Web application
├─ mobile/           # Flutter - iOS + Android app
├─ notify/           # Notification service + RabbitMQ workers
├─ infra/            # Docker Compose, Terraform, GitHub Actions
├─ android/          # (reference only - Flutter targets Android
automatically)
├─ ios/              # (reference only - Flutter targets iOS
automatically)
├─ .env.example      # Root-level env var template
└─ docker-compose.yml # Quick-start local development stack
```

Related Repositories

Repository	Description
infra	Infrastructure configuration
backend	Backend services
frontend	Frontend application
notify	Notification service
ios	iOS application (Flutter)
android	Android application (Flutter)